Agentic Patterns

From Tool Use to Governed Agency

Bryan Bischof

25 Mar 2026

Today's Journey

Unit 5 gave the model tools. Unit 9 adds **agency**: the model owns multi-step workflow decisions under constraints.

Through-Line

The key engineering move is from "tool use" to **harnessed runtime control**.

Core frameworks

- Ladder of agency
- Determinism $\times$ agency matrix
- P^3: Promise, Proof, Production

Practical goals

- Place real products in the matrix
- Level systems up safely
- Evaluate trajectories, not just answers

Outline

1. Framing and Recap
2. From Tools to Agent Loops
3. Agent as State Machine
4. Harness Engineering
5. Practical Harness Primitives
6. Subagents, Skills, and Retrieval
7. Frontier Directions
8. Proof and Production
9. Exercise and Close

Three Compatible Definitions of an Agent

1. **Classical AI:** an agent perceives and acts in an environment.
2. **RL/control:** an agent makes sequential decisions from feedback over time.
3. **Modern engineering:** an agent executes workflows with bounded independence using tools and guardrails.

Course definition

An agent is an LLM-driven control policy that observes state, chooses actions, updates behavior from outcomes, and runs under harness constraints until a verifiable exit condition is met.

Ladder of Agency (Units 5, 7, 8)

1. **Text output**: describe what should happen
2. **Structured output**: executable intent
3. **Tool-use loop**: thought → action → observation
4. **MCP**: interoperable tools and resources
5. **Subagents + skills**: delegation and reusable workflows

Unit 5 carryover

Loop-check discipline, determinism ladder, and scale primitives (MCP/subagents/skills) are the foundation for Unit 9.

Determinism × Agency Matrix

Axes

- **Determinism**: how constrained, testable, and auditable each step is
- **Agency**: how much workflow control the system owns over time

	Low determinism	**High determinism**
Low agency	Q1: Advisory explorers	Q2: Constrained copilots
High agency	Q3: Autonomous explorers	Q4: Operational agents

Placement is deployment-dependent: permissions, approvals, and runtime controls can move the same product across quadrants.

Matrix with Real Product Anchors

Q1 Advisory explorers

- Notion AI, Jasper
- Canva Magic Design, Adobe Firefly
- ChatGPT (chat), Perplexity (search)

Q2 Constrained copilots

- GitHub Copilot
- Claude Code / Cursor (approval-first)
- Hex Notebook Agent
- Nuance DAX Copilot, Zendesk AI Copilot
- Rossum Copilot, UiPath Doc Understanding

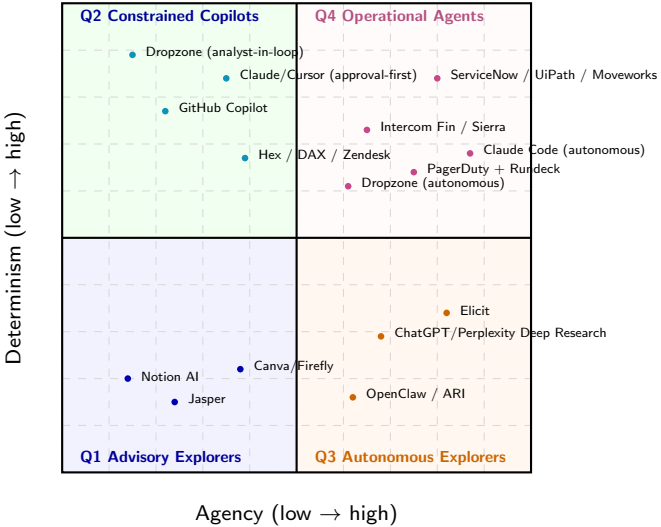
Q3 Autonomous explorers

- ChatGPT Deep Research
- Perplexity Deep Research
- OpenClaw (bounded-or-not by deployment)
- You.com ARI, Elicit

Q4 Operational agents

- Intercom Fin, Sierra AI
- ServiceNow AI Agents, UiPath Agent Builder
- Moveworks, PagerDuty + Rundeck
- Dropzone AI (autonomous triage mode)
- Claude Code (autonomous run mode)

Agency × Determinism Product Map



Q1: Advisory Explorers

Low agency, lower determinism
Human-owned execution, model-assisted exploration

Q1 Advisory Explorers: Content and Ideation (Part 1)

In Q1, the system expands options, but the user keeps execution authority.

- **Notion AI** drafts summaries and first-pass docs; teams choose what to publish.
- **Jasper** generates campaign copy variants for marketers to review and select.

Q1 Advisory Explorers: Content and Ideation (Part 2)

- **Canva Magic Design** proposes design directions; designers refine final layouts.
- **Adobe Firefly** generates visual concepts used as starting points, not final production.

Q1 Advisory Explorers: Search and Analysis Examples

- **ChatGPT (standard chat)** supports exploratory analysis and alternative framing without executing external side effects.
- **Perplexity (search mode)** returns source-backed answers, then humans decide what action to take.

Best-fit user stories

"Draft three GTM narratives from this interview set."
"Compare the top approaches and flag open questions for review."

Level-up path: Q1 → Q2 by adding schemas, citations, and reviewer checkpoints.

Q1 Deep Dive: Workflow Anatomy

Representative flow (Notion AI / ChatGPT / Perplexity)

1. User provides objective and context bundle.
2. Model proposes options, summaries, and framing alternatives.
3. Human compares options against external constraints.
4. Human executes the decision in downstream systems.

Why this matters

Q1 is often the highest-leverage entry point: low integration risk, fast time-to-value, and clear human accountability for side effects.

Q1 Deep Dive: Failure Modes and Upgrades

Failure mode	What it looks like	Upgrade path
Vague synthesis	Generic recommendations	Require objective + constraints template
Source weakness	Confident but weak evidence	Add citation requirement + evidence checklist
Option overload	Too many low-signal ideas	Force top-3 ranked options with tradeoffs

Teaching point: Q1 quality improves quickly with structure, even before adding automation.

Q1 Prompt Examples You Can Ask (Part 1)

- **Notion AI:** "Summarize these meeting notes into decisions, owners, and open questions. Keep unknowns explicit."
- **Jasper:** "Generate three launch-email variants for CFO, engineering lead, and operations manager personas."

Q1 Prompt Examples You Can Ask (Part 2)

- **Canva/Firefly:** "Create five visual concept directions for a trust-focused fintech landing page. Keep copy placeholders only."
- **ChatGPT/Perplexity:** "Compare top approaches for SOC triage automation and cite sources with publication dates."

Q2: Constrained Copilots

Higher determinism, human-in-the-loop control
Structured recommendations and approved execution

Q2 Constrained Copilots: Engineering and Analytics

In Q2, the system can propose actions in workflow context, but a human approves execution.

- **GitHub Copilot** suggests code and tests; developers decide what to apply.
- **Claude Code** / **Cursor** (approval-first) propose edits and commands before execution.
- **Hex Notebook Agent** drafts SQL/Python notebook steps; analysts validate outputs before publishing decisions.

Q2 Constrained Copilots: Service and Operations (Part 1)

- **Nuance DAX Copilot** drafts clinical notes; clinicians sign off.
- **Zendesk AI Copilot** suggests replies and actions; agents approve.

Q2 Constrained Copilots: Service and Operations (Part 2)

- **Dropzone AI** (analyst-in-loop mode) drafts investigations; SOC analysts choose containment actions.
- **Rossum Copilot** extracts document fields; operators verify uncertain values.

Level-up path: Q2 $\rightarrow$ Q4 once approval outcomes are stable and policy checks can be encoded as runtime gates.

Q2 Deep Dive: Workflow Anatomy

Representative flow (Copilot / DAX / Zendesk Copilot)

1. System reads local task state (ticket, note, PR, form).
2. Model drafts structured recommendation or action proposal.
3. Policy checks validate format, evidence, and confidence thresholds.
4. Human approves, edits, or rejects before side effects occur.

Control boundary

Decision support is automated; execution authority remains human.

Q2 Deep Dive: Readiness for Q4 Promotion

- **Approval agreement:** high reviewer acceptance for repeated task classes.
- **Low correction cost:** rejected outputs are cheap and rare.
- **Stable policy encoding:** approval logic can be codified as deterministic runtime checks.
- **Trace quality:** each recommendation has auditable evidence.

Do not promote early

If reviewers frequently rewrite or dispute core reasoning, keep the system in Q2 and improve context, schema, or tool boundaries first.

Q2 Prompt Examples You Can Ask (Part 1)

- **GitHub Copilot / Claude Code:** "Propose a refactor for this module, include tests, and explain tradeoffs before any apply step."
- **Hex Notebook Agent:** "Draft SQL and Python to segment churn risk by cohort, then list assumptions for analyst review."

Q2 Prompt Examples You Can Ask (Part 2)

- **Nuance DAX Copilot:** "Draft the SOAP note from this transcript and flag uncertain clinical details for clinician confirmation."
- **Zendesk Copilot:** "Suggest a response using policy section IDs and confidence scores; do not send without approval."

Q3: Autonomous Explorers

Higher agency with exploratory outcomes
Autonomous search loops, human downstream decisions

Q3 Autonomous Explorers: Research Loops

In Q3, the system owns multi-step exploration and synthesis, while humans keep final operational authority.

- **ChatGPT Deep Research** runs iterative web research and returns a cited report.
- **Perplexity Deep Research** autonomously gathers sources and compiles synthesis.
- **You.com ARI** performs iterative research passes and returns structured findings.

Q3 Autonomous Explorers: Boundary-Sensitive Cases

- **Elicit** searches literature and extracts evidence fields for human evaluation.
- **OpenClaw** orchestrates multi-step tasks; quadrant placement depends on permissions and controls.

Boundary condition

If these systems can write into high-impact production systems, they should be treated as Q4 risk profiles and governed accordingly.

Q3 Deep Dive: Workflow Anatomy

Representative flow (Deep Research / ARI / Elicit)

1. Agent plans a multi-hop research strategy from the query.
2. Agent iterates: retrieve, evaluate source quality, branch, and merge.
3. Agent compacts findings into milestone summaries with citations.
4. Human consumes report and decides operational follow-up.

Key distinction

Q3 owns exploration policy, not high-impact execution policy.

Q3 Deep Dive: Drift Controls for Long Runs

- **Checkpoint cadence:** require summary snapshots every N steps.
- **Context compaction:** keep evidence tables, drop stale chatter.
- **Branch budget:** cap speculative branches to avoid search bloat.
- **Citation gates:** block final output without source linkage.

Failure pattern to watch: long autonomous loops that look active but stop adding net new evidence.

Q3 Prompt Examples You Can Ask (Part 1)

- **Deep Research:** "Research build-vs-buy options for customer support automation, cite sources, and include a risk matrix."
- **Perplexity Deep Research:** "Produce a market map of enterprise agent platforms with pricing signals and source links."

Q3 Prompt Examples You Can Ask (Part 2)

- **You.com ARI:** "Run a multi-pass analysis of incident response benchmarks by industry and summarize deltas."
- **Elicit/OpenClaw:** "Gather evidence for claims about RAG eval methods; mark weakly supported claims as provisional."

Q4: Operational Agents

High agency plus high determinism
Bounded autonomous execution under policy and audit

Q4 Operational Agents: Customer and Enterprise (Part 1)

In Q4, the system executes bounded workflows end-to-end under policy, observability, and recovery controls.

- **Intercom Fin** resolves support requests directly and escalates by policy.
- **Sierra AI agents** take customer-service actions in connected systems under guardrails.

Q4 Operational Agents: Customer and Enterprise (Part 2)

- **ServiceNow AI Agents** execute enterprise workflows with approval and audit controls.
- **UiPath Agent Builder** coordinates agents, RPA, and humans in operational flows.

Q4 Operational Agents: Security and Incidents (Part 1)

- **Moveworks** executes IT/HR service operations through integrated enterprise tools.
- **PagerDuty + Rundeck** runs bounded remediation from predefined runbooks.

Q4 Operational Agents: Security and Coding (Part 2)

- **Dropzone AI** (autonomous triage mode) investigates and classifies alerts at SOC scale.
- **Claude Code** in autonomous mode plans, edits, tests, and iterates to explicit exit criteria.

Deployment rule: move right first (determinism), then up (agency), and expand scope only after trajectory and policy evals are passing.

Q4 Deep Dive: Workflow Anatomy

Representative flow (Intercom Fin / ServiceNow / UiPath)

1. Agent classifies case and selects bounded workflow template.
2. Pre-action policy gate validates permissions and invariants.
3. Agent executes side effects through approved tools and runbooks.
4. Post-action verification confirms state and closes or escalates.

Operational requirement

Recovery logic and escalation paths are first-class, not optional.

Q4 Deep Dive: Production Controls and Metrics

Control area	Minimum control	Observed metric
Policy safety	Immutable deny/allow constraints	policy violation rate
Execution quality	Pre/post action validation	successful completion rate
Reliability	Retry + checkpoint + fallback owner	recovery time
Governance	Traceable decisions and tool calls	audit pass rate

Promotion rule: expand workflow scope only when all four control areas hold under production-like load.

Q4 Prompt Examples You Can Ask (Part 1)

- **Intercom Fin / Sierra:** "Resolve billing tickets under policy v4; escalate if confidence is below 0.85 or policy conflict exists."
- **ServiceNow / UiPath:** "Run the onboarding workflow for this user set, require approval at permission grant step, and log all actions."

Q4 Prompt Examples You Can Ask (Part 2)

- **Dropzone (autonomous):** "Triage P1 alerts, execute approved containment runbooks, and escalate any endpoint quarantine failures."
- **Claude Code (autonomous):** "Implement this issue end-to-end; run tests, stop on failing security checks, and open a PR summary."

Context from Unit 7 and Unit 8 in the Matrix

Unit 7 (Retrieval)

- Q1: one-shot search and synthesis
- Q2: retrieval with schema/citation constraints
- Q3: iterative multi-hop exploration
- Q4: retrieval in governed production workflows

Unit 8 (Context Engineering)

- moving right: improve determinism via context controls
- moving up: preserve state quality under longer autonomous runs

Context Engineering Priorities by Quadrant

Quadrant	Typical context challenge	Primary focus
Q1	Over-broad context, generic output	Fast breadth with explicit unknowns and light structure
Q2	Format-compliant output, weak grounding	Precision/citations/schema checks and reviewer-visible evidence
Q3	Drift across long exploration loops	Milestone summaries, resets, compaction, checkpointed research state
Q4	Context mistakes become side-effect mistakes	Policy-first context, immutable constraints, pre-action validation blocks

What Unit 9 Adds

1. **Capability demos → workflow ownership**
2. **Output quality → system quality**
3. **Ad hoc context → engineered context**
4. **Tool calls → operating models**

Level-up sequence

Move right first (determinism), then up (agency), then widen scope only after proof and controls hold.

Outline

1. Framing and Recap
2. From Tools to Agent Loops
3. Agent as State Machine
4. Harness Engineering
5. Practical Harness Primitives
6. Subagents, Skills, and Retrieval
7. Frontier Directions
8. Proof and Production
9. Exercise and Close

The Transition

Tool use: "call this function now."
Agentic behavior: "decide what to do next until done."

Canonical loop

Thought $\rightarrow$ Action $\rightarrow$ Observation $\rightarrow$ Thought $\rightarrow \cdots \rightarrow$ Final output

ReAct provides the loop skeleton. Harness design determines where that loop can safely operate in the matrix.

ReAct-like $\neq$ Agent

ReAct-like but often not agents

- support copilot in ticket UI
- SOC assistant recommending containment
- IDE coding assistant proposing edits/tests

Typically agentic

- refund resolution agent with policy gates
- incident remediation agent with runbook execution
- long-running coding agent with test-based exit criteria

Copilot vs Agent Diagnostic

Three questions decide copilot vs agent

1. Who decides the next action?
2. Who executes side effects?
3. Who decides completion criteria are met?

Decision rule

If the runtime owns all three under explicit constraints, treat it as an agentic system.

Copilot vs Agent: Worked Classification

System	Who decides next?	Who decides done?	Agent?
Support copilot	Human rep	Human rep	No
SOC copilot	Human analyst	Human analyst	No
IDE assistant	Developer	Developer	No
Refund agent	Agent policy	Agent + policy	Yes
Incident agent	Agent policy	Agent verifies recovery	Yes
Coding agent	Agent loop	Milestones + tests	Yes

How Systems Level Up in Agency

Support path (Q1 → Q2 → Q4)

- Q1: draft replies and summarize tickets
- Q2: enforce templates/citations before send
- Q4: bounded autonomous case resolution + escalation

Coding path (Q2 → Q4)

- Q2: suggest edits/tests with human approval per step
- Q4: agent plans, edits, tests, fixes, checkpoints, resumes

Outline

1. Framing and Recap
2. From Tools to Agent Loops
3. Agent as State Machine
4. Harness Engineering
5. Practical Harness Primitives
6. Subagents, Skills, and Retrieval
7. Frontier Directions
8. Proof and Production
9. Exercise and Close

State Machines Make Agency Legible

Without a state model, behavior is opaque. With a state model, behavior is inspectable and debuggable.

- **State**: goal, plan, memory, artifacts
- **Action policy**: what is legal/useful now
- **Transition**: how state updates on outcomes
- **Termination**: success, failure, timeout, stop

Q3/Q4 systems require explicit transitions to avoid drift and loop failure.

Operational State Nodes

1. Plan
2. Act
3. Observe
4. Verify
5. Commit checkpoint
6. Continue or stop

As agency increases, these nodes become runtime artifacts rather than implied prompt behavior.

Checkpointing and Recovery

Long-running agents fail for practical reasons: crashes, stale context, tool outages, transient permissions.

Checkpoint artifacts

- current plan and completed milestones
- latest verified outputs
- open risks and next actions
- tool-call history pointer
- pending approval requests

Q2 often needs review-queue checkpoints; Q4 needs durable resumability.

Outline

1 Framing and Recap

2 From Tools to Agent Loops

3 Agent as State Machine

4 Harness Engineering

5 Practical Harness Primitives

6 Subagents, Skills, and Retrieval

7 Frontier Directions

8 Proof and Production

9 Exercise and Close

Harness Engineering: What Is New in Unit 9?

What is new

Earlier units optimize *single steps*. Unit 9 optimizes *multi-step control of a workflow over time*.

- **Tool-use (Unit 5):** can call tools
- **RAG (Unit 7):** can fetch evidence
- **Context engineering (Unit 8):** can shape model inputs
- **Harness engineering (Unit 9):** decides what happens next, under policy, until done

Harness = The Agent Runtime

Layer	Role
Model	reason, draft plans, choose candidate actions
Harness	enforce policy, manage state, orchestrate loop, verify outcomes
Environment	files, APIs, browser, databases, systems of record

Why important

Agent failures are usually harness failures: bad control logic, weak gates, or missing recovery paths.

Harness Component 1: Runtime Policy

What is new

- policy moves from prompt text into executable runtime checks
- each action is validated before side effects occur

Why it is important

- prevents unsafe or out-of-scope actions
- creates auditable, deterministic behavior at higher agency

Harness Component 2: State + Orchestration

What is new

- explicit workflow state (goal, step, artifacts, checkpoints)
- loop controller handles retries, branching, and stop conditions

Why it is important

- enables long-running work without losing progress
- makes behavior debuggable and resumable after failures

Harness Component 3: Context + Delegation

What is new

- context is assembled per step, not dumped once
- subagents get scoped context and bounded responsibilities

Why it is important

- reduces context drift in multi-step runs
- allows higher agency without overwhelming one orchestrator

Harness Component 4: Verification + Observability

What is new

- completion is validated by tests, invariants, and policy checks
- traces capture trajectory, tool calls, and outcomes

Why it is important

- avoids "looks good" outputs that fail in reality
- makes root-cause analysis and production governance possible

Compare the Building Blocks

Part 1: capability and grounding

- **Tool-use (Unit 5):** executes capabilities
- **RAG (Unit 7):** grounds answers in evidence

Not enough alone

Tool-use + RAG still do not provide multi-step workflow ownership, recovery policy, or governed completion logic.

Compare the Building Blocks (Cont.)

Part 2: control quality and runtime control

- **Context engineering (Unit 8):** improves input quality per step
- **Harness engineering (Unit 9):** owns next-step policy over time

Integration point

Harness engineering does not replace tools, RAG, or context engineering. It composes them into a governed runtime.

How It Comes Together to Become Agentic

1. **Tool-use** gives actions
2. **RAG** gives grounded information
3. **Context engineering** gives clean step inputs
4. **Harness engineering** gives control policy over time

Agentic threshold

A system becomes agentic when the runtime, not the human, owns next-step policy and completion logic under explicit constraints.

Harness Upgrade Map in the Matrix

Move right (higher determinism)

- stronger runtime policy and verification gates
- stricter evidence and action preconditions
- clearer audit traces and rollback paths

Move up (higher agency)

- richer state models and orchestration loops
- broader delegated execution under bounded authority
- stronger recovery and escalation logic

Cheat Sheet: Q1/Q2 Progression

- **Q1 Advisory**
  - Tool-use: optional
  - RAG: basic lookup/synthesis
  - Context: prompt hygiene
  - Harness: light loop control
- **Q2 Copilot**
  - Tool-use: task-scoped actions
  - RAG: cited grounded drafts
  - Context: schema + evidence context
  - Harness: approval-first orchestration

Cheat Sheet: Q3/Q4 Progression

- **Q3 Explorer**
  - Tool-use: iterative action chains
  - RAG: multi-hop evidence loops
  - Context: compaction + checkpoints
  - Harness: autonomous loop policy
- **Q4 Operator**
  - Tool-use: bounded production actions
  - RAG: governed retrieval
  - Context: immutable policy context
  - Harness: end-to-end execution policy

Progression rule

Move right first (determinism), then up (agency).

Outline

1 Framing and Recap

2 From Tools to Agent Loops

3 Agent as State Machine

4 Harness Engineering

5 Practical Harness Primitives

6 Subagents, Skills, and Retrieval

7 Frontier Directions

8 Proof and Production

9 Exercise and Close

Six Primitives That Matter in Production

1. Durable workspace and externalized state
2. Code execution as a universal action substrate
3. Context engineering inside the loop
4. Verification loops
5. Human-in-the-loop gates
6. Rate limits, retries, and stop conditions

These are the controls that convert autonomy into governable operations.

Practical Level-Up Examples

Coding

- Q2: suggestions + per-step approvals
- Q4: autonomous milestone loop + tests + resumability

Support

- Q2: constrained draft replies + agent approval
- Q4: bounded autonomous resolution + escalation

Approval gates are often the bridge from Q2 to Q4.

Outline

1. Framing and Recap
2. From Tools to Agent Loops
3. Agent as State Machine
4. Harness Engineering
5. Practical Harness Primitives
6. Subagents, Skills, and Retrieval
7. Frontier Directions
8. Proof and Production
9. Exercise and Close

Tools vs Skills vs Subagents

- **Tool**: low-level action with narrow contract
- **Skill**: reusable workflow package
- **Subagent**: delegated autonomous worker with scoped context

Matrix interpretation:

- tools increase capability breadth
- skills increase repeatability (determinism)
- subagents increase agency with context boundaries

Retrieval as Agent Capability

Practical pattern:

1. Orchestrator defines objective
2. Retrieval subagent runs multi-hop search
3. Synthesis subagent composes evidence
4. Orchestrator verifies citations/gaps

This is a common path from Q2 retrieval assistants to Q4 retrieval-backed operators: add iterative planning, evidence checks, and escalation logic.

Outline

1. Framing and Recap
2. From Tools to Agent Loops
3. Agent as State Machine
4. Harness Engineering
5. Practical Harness Primitives
6. Subagents, Skills, and Retrieval
7. Frontier Directions
8. Proof and Production
9. Exercise and Close

Three Frontier Directions

1. **Code-native runtimes**: program against runtime state
2. **Recursive context access**: navigate context programmatically
3. **Automatic harness synthesis**: search for better constraints

Frontier Matrix Effects

- **Code-native runtime**: raises agency quickly; requires stronger safety and verification.
- **Recursive context access**: improves determinism as loop depth increases.
- **Harness synthesis**: can move systems right via learned validators and policy tuning.

Outline

1 Framing and Recap

2 From Tools to Agent Loops

3 Agent as State Machine

4 Harness Engineering

5 Practical Harness Primitives

6 Subagents, Skills, and Retrieval

7 Frontier Directions

8 Proof and Production

9 Exercise and Close

Proof = Trajectory Evaluation

Final-answer grading is insufficient for agents.

Evaluate at least four layers:

1. answer correctness
2. trajectory correctness
3. environment-state correctness
4. policy compliance

In high-agency systems, retrieval/context/tool errors become action errors.

Proof Depth by Quadrant: Q1/Q2

- **Q1**: usefulness, source quality, option diversity
- **Q2**: schema compliance, reviewer agreement, policy adherence

Interpretation

Low-agency tiers emphasize output quality and human reviewer reliability.

Proof Depth by Quadrant: Q3/Q4

- **Q3**: trajectory stability and evidence consistency across loops
- **Q4**: all prior checks plus environment-state correctness, rollback safety, and operational reliability

Release rule

Promote to higher agency only after consistently meeting proof criteria at the current quadrant.

P^3 for Unit 9

Promise

- complete real jobs end-to-end under constraints

Proof

- trajectory validity, environment outcomes, policy compliance

Production

- checkpointing, approvals, budgets, retries, resumability

Outline

1 Framing and Recap

2 From Tools to Agent Loops

3 Agent as State Machine

4 Harness Engineering

5 Practical Harness Primitives

6 Subagents, Skills, and Retrieval

7 Frontier Directions

8 Proof and Production

9 Exercise and Close

Exercise: Diagnose the Harness

Use one task and three traces:

1. Naive ReAct loop
2. Harnessed loop with subagent isolation
3. Verifier-augmented loop

For each trace, identify:

- model failure vs harness failure
- missing invariant
- checkpoint or approval gate placement
- trace signal that would have caught failure earlier

Key Takeaways

1. Agents are runtime systems, not prompt templates.
2. ReAct is the loop baseline; harness design determines reliability.
3. Move right before moving up: strengthen determinism before agency.
4. Context, state, and delegation are core control surfaces.
5. Trajectory-first evaluation is required for autonomous behavior.
6. Frontier progress is harness-centric.

Further Reading

Required

- Shankar & Husain, Chapter 8 (8.1–8.2)
- ReAct (Yao et al., 2022)

Strongly recommended

- OpenAI: Building Agents
- Anthropic: Effective harnesses/context engineering
- OpenAI Codex docs: AGENTS.md, Skills, MCP
- LangChain: Anatomy of an Agent Harness